

静态链接器是由像 GCC 这样的编译驱动程序调用的。它们将多个可重定位目标文件合并成一个单独的可执行目标文件。多个目标文件可以定义相同的符号，而链接器用来悄悄地解析这些多重定义的规则可能在用户程序中引入微妙的错误。

多个目标文件可以被连接到一个单独的静态库中。链接器用库来解析其他目标模块中的符号引用。许多链接器通过从左到右的顺序扫描来解析符号引用，这是另一个引起令人迷惑的链接时错误的来源。

加载器将可执行文件的内容映射到内存，并运行这个程序。链接器还可能生成部分链接的可执行目标文件，这样的文件中有对定义在共享库中的例程和数据的未解析的引用。在加载时，加载器将部分链接的可执行文件映射到内存，然后调用动态链接器，它通过加载共享库和重定位程序中的引用来完成链接任务。

被编译为位置无关代码的共享库可以加载到任何地方，也可以在运行时被多个进程共享。为了加载、链接和访问共享库的函数和数据，应用程序也可以在运行时使用动态链接器。

参考文献说明

在计算机系统文献中并没有很好地记录链接。因为链接是处在编译器、计算机体系结构和操作系统的交叉点上，它要求理解代码生成、机器语言编程、程序实例化和虚拟内存。它没有恰好落在某个通常的计算机系统领域中，因此这些领域的经典文献并没有很好地描述它。然而，Levine 的专著提供了有关这个主题的很好的一般性参考资料[69]。[54]描述了 ELF 和 DWARF(对 .debug 和 .line 节内容的规范)的原始 IA32 规范。[36]描述了对 ELF 文件格式的 x86-64 扩展。x86-64 应用二进制接口(ABI)描述了编译、链接和运行 x86-64 程序的惯例，其中包括重定位和位置无关代码的规则[77]。

家庭作业

- 7.6 这道题是关于图 7-5 的 m.o 模块和下面的 swap.c 函数版本的，该函数计算自己被调用的次数：

```

1  extern int buf[];
2
3  int *bufp0 = &buf[0];
4  static int *bufp1;
5
6  static void incr()
7  {
8      static int count=0;
9
10     count++;
11 }
12
13 void swap()
14 {
15     int temp;
16
17     incr();
18     bufp1 = &buf[1];
19     temp = *bufp0;
20     *bufp0 = *bufp1;
21     *bufp1 = temp;
22 }
```

对于每个 swap.o 中定义和引用的符号，请指出它是否在模块 swap.o 的 .symtab 节中有符号表条目。如果是这样，请指出定义该符号的模块(swap.o 或 m.o)、符号类型(局部、全局或外部)以及它在模块中所处的节(.text、.data 或 .bss)。

符号	swap.o.symtab条目?	符号类型	定义符号的模块	节
buf				
bufp0				
bufp1				
swap				
temp				
incr				
count				

* 7.7 不改变任何变量名字，修改 7.6.1 节中的 bar5.c，使得 foo5.c 输出 x 和 y 的正确值(也就是整数 15213 和 15212 的十六进制表示)。

* 7.8 在此题中，REF(x,i)→DEF(x,k)表示链接器将任意对模块 i 中符号 x 的引用与模块 k 中符号 x 的定义相关联。在下面每个例子中，用这种符号来说明链接器是如何解析在每个模块中有多重定义的引用的。如果出现链接时错误(规则 1)，写“错误”。如果链接器从定义中任意选择一个(规则 3)，那么写“未知”。

```
A. /* Module 1 */           /* Module 2 */
   int main()               static int main=1[
   {                         int p2()
   {                         {
   }                         }
   }
```

- (a) REF(main.1) → DEF(_____)
- (b) REF(main.2) → DEF(_____)

```
B. /* Module 1 */           /* Module 2 */
   int x;                   double x;
   void main()              int p2()
   {                         {
   {                         }
   }
```

- (a) REF(x.1) → DEF(_____)
- (b) REF(x.2) → DEF(_____)

```
C. /* Module 1 */           /* Module 2 */
   int x=1;                 double x=1.0;
   void main()              int p2()
   {                         {
   {                         }
   }
```

- (a) REF(x.1) → DEF(_____)
- (b) REF(x.2) → DEF(_____)

* 7.9 考虑下面的程序，它由两个目标模块组成：

```
1  /* foo6.c */              1  /* bar6.c */
2  void p2(void);            2  #include <stdio.h>
3                             3
4  int main()                4  char main;
5  {                          5
6      p2();                  6  void p2()
7      return 0;              7  {
8  }                          8      printf("0x%x\n", main);
                              9  }
```

当在 x86-64 Linux 系统中编译和执行这个程序时，即使函数 p2 不初始化变量 main，它也能打印字符串“0x48\n”并正常终止。你能解释这一点吗？

** 7.10 a 和 b 表示当前路径中的目标模块或静态库，而 a→b 表示 a 依赖于 b，也就是说 a 引用了一个 b 定义的符号。对于下面的每个场景，给出使得静态链接器能够解析所有符号引用的最小的命令行(即含有最少数量的目标文件和库参数的命令)。

- A. p.o→libx.a→p.o
B. p.o→libx.a→liby.a 和 liby.a→libx.a
C. p.o→libx.a→liby.a→libz.a 和 liby.a→libx.a→libz.a
- ** 7. 11 图 7-14 中的程序头部表明数据段占用了内存中 0x230 个字节。然而，其中只有开始的 0x228 字节来自可执行文件的节。是什么引起了这种差异？
- ** 7. 12 考虑目标文件 m.o 中对函数 swap 的调用(作业题 7. 6)。

```
9:  e8 00 00 00 00      callq  e <main+0xe>      swap()
```

具有如下重定位条目：

```
r.offset = 0xa  
r.symbol = swap  
r.type   = R_X86_64_PC32  
r.addend = -4
```

A. 假设链接器将 m.o 中的 .text 重定位到地址 0x4004e0，把 swap 重定位到地址 0x4004f8。那么 callq 指令中对 swap 的重定位引用的值应该是什么？
B. 假设链接器将 m.o 中的 .text 重定位到地址 0x4004d0，把 swap 重定位到地址 0x400500。那么 callq 指令中对 swap 的重定位引用的值应该是什么？
- ** 7. 13 完成下面的任务将帮助你更熟悉处理目标文件的各种工具。
A. 在你的系统上，lib.c 和 libm.a 的版本中包含多少目标文件？
B. gcc-Og 产生的可执行代码与 gcc-Og-g 产生的不同吗？
C. 在你的系统上，GCC 驱动程序使用的是什么共享库？
- 练习题答案
7. 1 这道练习题的目的是帮助你理解链接器符号和 C 变量及函数之间的关系。注意 C 的局部变量 temp 没有符号表条目。
- | 符号 | .symtab 条目? | 符号类型 | 在哪个模块中定义 | 节 |
|-------|-------------|------|----------|--------|
| buf | 是 | 外部 | main.o | .data |
| bufp0 | 是 | 全局 | swap.o | .data |
| bufp1 | 是 | 全局 | swap.o | COMMON |
| swap | 是 | 全局 | swap.o | .text |
| temp | 否 | — | — | — |
7. 2 这是一个简单的练习，检查你对 Unix 链接器解析在一个以上模块中有定义的全局符号时所使用规则的理解。理解这些规则可以帮助你避免一些讨厌的编程错误。
A. 链接器选择定义在模块 1 中的强符号，而不是定义在模块 2 中的弱符号(规则 2)：
 (a) REF(main.1) → DEF(main.1)
 (b) REF(main.2) → DEF(main.1)
B. 这是一个错误，因为每个模块都定义了一个强符号 main(规则 1)。
C. 链接器选择定义在模块 2 中的强符号，而不是定义在模块 1 中的弱符号(规则 2)：
 (a) REF(x.1) → DEF(x.2)
 (b) REF(x.2) → DEF(x.2)

7. 3 在命令行中以错误的顺序放置静态库是造成令许多程序员迷惑的链接器错误的常见原因。然而，一旦你理解了链接器是如何使用静态库来解析引用的，它就相当简单易懂了。这个小练习检查了你对这个概念的理解：
A. linux> gcc p.o libx.a
B. linux> gcc p.o libx.a liby.a
C. linux> gcc p.o libx.a liby.a libx.a

7. 4 这道题涉及的是图 7-12a 中的反汇编列表。目的是让你练习阅读反汇编列表，并检查你对 PC 相对